

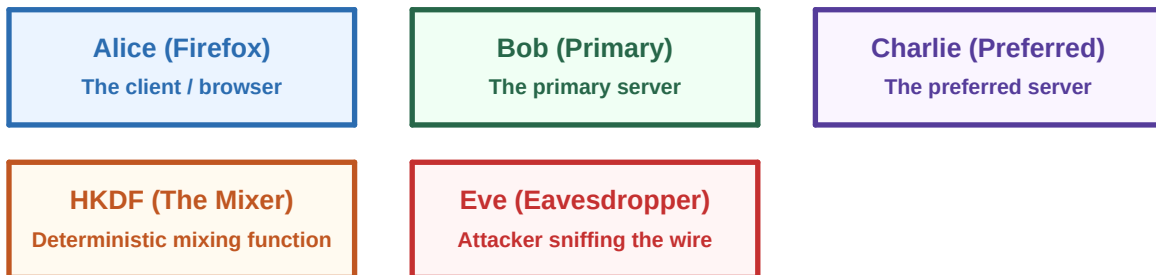
How QUIC Secrets Are Built

A Visual, Step-by-Step Guide

Who Gives What, When, and How It All Mixes Together

Reference Document • June 2026

Cast of Characters



1. The Analogy: Paint Mixing

Before diving into crypto, here's the intuition. Imagine secrets as paint colors:

Alice has a SECRET paint color: [PRIVATE BLUE] (only she knows)
Bob has a SECRET paint color: [PRIVATE GREEN] (only he knows)

They agree on a PUBLIC base: [YELLOW] (everyone knows)

Step 1: Alice mixes her secret with the base:
[YELLOW] + [PRIVATE BLUE] = [TEAL] → sends TEAL to Bob (public)

Step 2: Bob mixes his secret with the base:
[YELLOW] + [PRIVATE GREEN] = [LIME] → sends LIME to Alice (public)

Step 3: Each mixes their secret with what they received:
Alice: [LIME] + [PRIVATE BLUE] = [DARK OLIVE] ← shared secret!
Bob: [TEAL] + [PRIVATE GREEN] = [DARK OLIVE] ← same color!

Eve sees: [YELLOW], [TEAL], [LIME]
Eve CANNOT unmix paint to get [PRIVATE BLUE] or [PRIVATE GREEN]
Eve CANNOT compute [DARK OLIVE]

This is Diffie-Hellman. In QUIC, the "paint" is math on elliptic curves.

The real QUIC version uses X25519 (an elliptic curve). "Mixing" is point multiplication. "Unmixing" would require solving the discrete logarithm problem -- computationally impossible.

2. Meet the Ingredients (Who Provides What)

Ingredient	Who Provides It	Public?	Example Value	Purpose
DCID	Alice picks randomly	Yes (in the clear)	0x8394c8f03e515708	Seed for Initial keys only
QUIC v1 Salt	Nobody (hardcoded in RFC)	Yes (public constant)	0x38762cf7f559...0a	Adds entropy to Initial key derivation
Client Random	Alice generates	Yes (in ClientHello)	0xa1b2c3d4... (32 bytes)	Prevents replay; part of transcript
Server Random	Bob generates	Yes (in ServerHello)	0xe5f6a7b8... (32 bytes)	Prevents replay; part of transcript
Alice's X25519 Private	Alice generates	NO! Secret!	0x[32 random bytes]	Her secret paint color
Alice's X25519 Public	Alice computes from private	Yes (in ClientHello)	0x358072d636... (32 bytes)	Her mixed paint (base + secret)
Bob's X25519 Private	Bob generates	NO! Secret!	0x[32 random bytes]	His secret paint color

Ingredient	Who Provides It	Public?	Example Value	Purpose
Bob's X25519 Public	Bob computes from private	Yes (in ServerHello)	0x9fd7ad6dcf... (32 bytes)	His mixed paint (base + secret)
Certificate	Bob (from CA)	Yes (sent in handshake)	[X.509 cert chain]	Proves Bob is who he claims
TLS Labels	Nobody (defined in RFC)	Yes (public strings)	"client in", "quic key"	Domain separators for HKDF

Only TWO things are truly secret: Alice's private key and Bob's private key. Everything else is public. Yet from these two secrets, an unbreakable shared secret is derived that no eavesdropper can compute.

3. Step-by-Step: Building the Secrets

Step 1: Alice Creates a Random Connection ID

Alice (Firefox) wants to connect. She picks a random Destination Connection ID. This will be used ONLY for the Initial packet encryption -- it provides no real security.

Alice picks: DCID = 0x8394c8f03e515708 (8 random bytes)

Alice sends this in the clear as the first byte of her first packet.
Eve can see it. Bob can see it. Everyone can see it.



Eve can also compute this! She saw the DCID on the wire, and the salt is public. That's why Initial packets are NOT truly encrypted -- they use keys anyone can derive.

Step 2: Initial Keys (For First Few Packets Only)

From the initial_secret, we derive separate keys for Alice→Bob and Bob→Alice:



Same secret + different label = different keys. The label acts like a "which door" selector. HKDF always gives the same output for the same inputs (deterministic).

From each Initial secret, derive 3 actual keys:

```
client_initial_secret → HKDF-Expand("quic key", 16) = AES key    [16 bytes]
                      → HKDF-Expand("quic iv", 12) = IV/nonce [12 bytes]
                      → HKDF-Expand("quic hp", 16) = HP key    [16 bytes]
```

Example (from RFC 9001 test vectors):

```
client_initial_secret = 0xc00cf151ca5be075ed0ebfb5c80323c4...
```

```
■ AES key = 0x1f369613dd76d5467730efcbe3b1a22d
```

```
■ IV      = 0xfa044b2f42a3fd3b46fb255c
```

```
■ HP key  = 0x9f50449e04a0e810283a1e9933adedd2
```

These keys encrypt the ClientHello and ServerHello. They contain the TLS key exchange data that will produce the REAL secrets in the next step.

Step 3: The Key Exchange (The Real Magic)

This is the most important step. Alice and Bob each generate a fresh, random X25519 key pair. They exchange public keys and independently compute the same shared secret.

3a. Alice generates her key pair

```
Alice (Firefox) generates:
private_key_alice = random 32 bytes = 0x49af42ba...
    ■■ NEVER sent anywhere. Dies when Firefox closes.

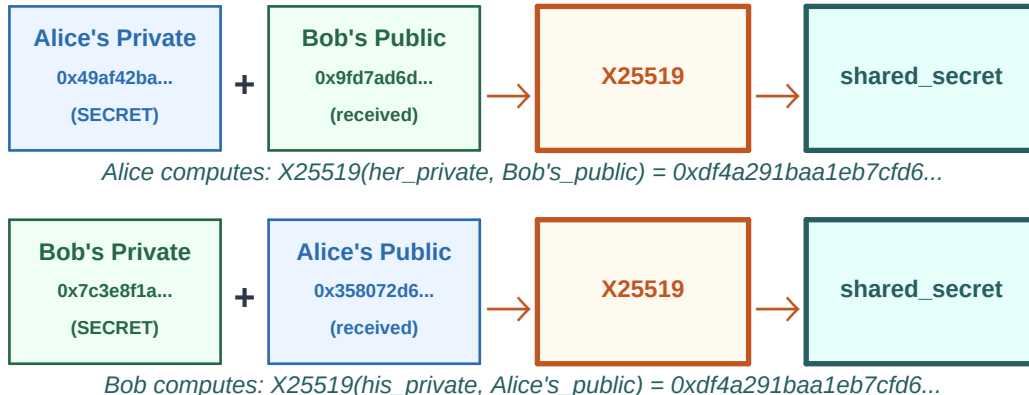
public_key_alice = X25519_basepoint_multiply(private_key_alice)
=
0x358072d6365880d1aeea329adf9121383851ed21a28e3b75e965d0d2cd166254
    ■■ Sent in ClientHello key_share extension (visible to Eve)
```

3b. Bob generates his key pair

```
Bob (Primary Server) generates:
private_key_bob = random 32 bytes = 0x7c3e8f1a...
    ■■ NEVER sent anywhere. Stays in server memory.

public_key_bob = X25519_basepoint_multiply(private_key_bob)
=
0x9fd7ad6dcff4298dd3f96d5b1b2af910a0535b1488d7f8fabb349a982880b615
    ■■ Sent in ServerHello key_share extension (visible to Eve)
```

3c. Both compute the shared secret INDEPENDENTLY

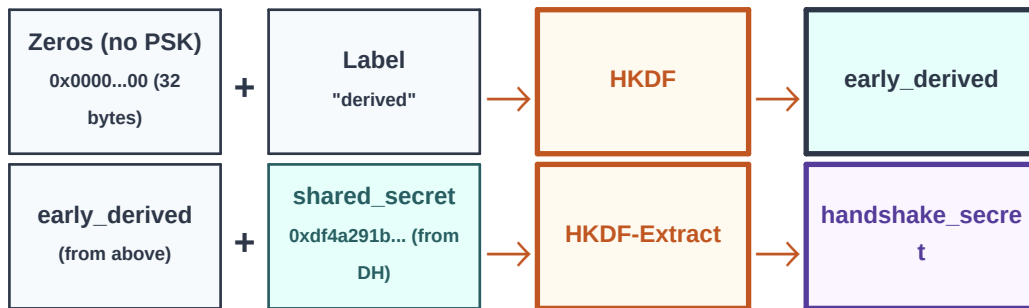


SAME RESULT! Both get 0xdf4a291baa1eb7cfd6... (32 bytes). This is the Diffie-Hellman miracle: $X25519(a_priv, b_pub) = X25519(b_priv, a_pub)$. Neither side needs the other's private key.

Eve sees BOTH public keys on the wire but CANNOT compute the shared secret. She would need either private key, which requires solving the Elliptic Curve Discrete Logarithm Problem -- computationally impossible.

Step 4: From Shared Secret to Handshake Keys

The shared secret goes through HKDF to produce handshake encryption keys. These encrypt the certificate exchange and handshake completion.



Now derive separate handshake traffic secrets for each direction. The transcript hash ensures these keys are bound to THIS specific handshake (prevents replays):

```
transcript_hash = SHA-256(ClientHello || ServerHello)
                  = 0x[hash of every byte exchanged so far]
                  This includes: randoms, public keys, SNI, ALPN, extensions, cipher suite

client_hs_secret = HKDF-Expand-Label(
    handshake_secret, "c hs traffic", transcript_hash, 32
)
server_hs_secret = HKDF-Expand-Label(
    handshake_secret, "s hs traffic", transcript_hash, 32
)

Each → AEAD key + IV + HP key (same 3-key derivation as Initial)
```

These keys encrypt: Bob's Certificate, CertificateVerify (proves he owns the cert), and both sides' Finished messages (proves no one tampered with the handshake).

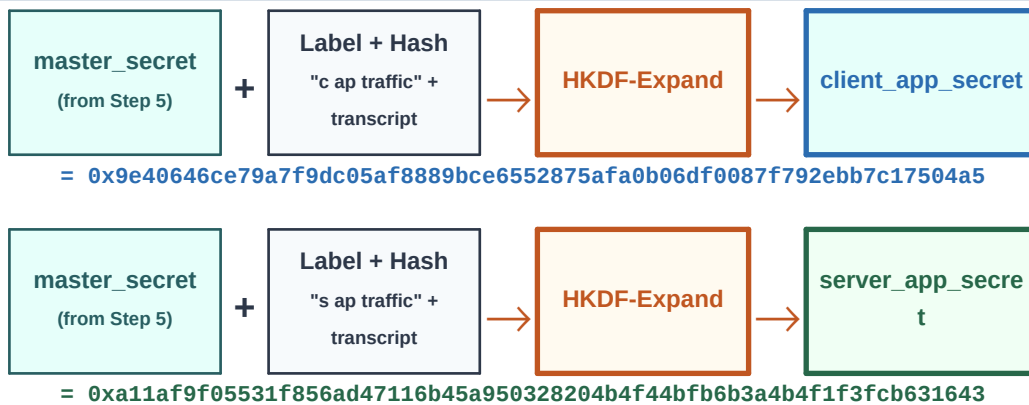
Step 5: From Handshake to Master Secret



Step 6: The Application Secrets (WHAT WE EXPORT!)

This is the final derivation. The master secret + full transcript hash produce the application traffic secrets. These are the 32-byte values we export for migration.

```
transcript_hash_full = SHA-256(  
  ClientHello || ServerHello || EncryptedExtensions ||  
  Certificate || CertificateVerify || Finished  
)  
= 0x[hash of the ENTIRE handshake -- every single byte]
```



THESE two 32-byte values are what we export to Charlie (the preferred server). They contain the distilled result of the entire handshake: Alice's randomness, Bob's randomness, the DH exchange, the transcript, everything. From these 64 bytes, Charlie can derive every encryption key needed.

Step 7: From Secrets to Actual Encryption Keys

Each 32-byte secret produces 3 keys through HKDF with different labels:

```
From server_app_secret (for encrypting server → client):  
  
HKDF-Expand-Label(secret, "quic key", 16)  
= 0xfd8c7da9de1b2da4d2ef9fd5188922d0      ← AES-128 key  
  
HKDF-Expand-Label(secret, "quic iv", 12)  
= 0x02f6180e4f4aa456d7e8a389              ← IV (nonce base)  
  
HKDF-Expand-Label(secret, "quic hp", 16)  
= 0xb7f6f021453e43b0b2c25989d0a31207      ← Header Protection key
```

Same inputs → same outputs. ALWAYS. This is why Charlie can reconstruct identical keys from just the 32-byte secret.

4. The Handoff: Bob Gives Charlie the Secrets

Now the migration happens. Bob (primary server) sends Charlie (preferred server) the application traffic secrets -- and Charlie rebuilds everything.

What Bob sends to Charlie (445 bytes):

Component	From Whom	Example	Size
server_app_secret	Bob derived from handshake	0xa11af9f05531f856...	32 bytes
client_app_secret	Bob derived from handshake	0x9e40646ce79a7f9d...	32 bytes
server_next_secret	Bob computed for key update	0x7b4f2e8c9a1d3f5e...	32 bytes
client_next_secret	Bob computed for key update	0x3c8b1d4e7f2a9085...	32 bytes
Cipher suite	Negotiated in handshake	0x1301 (AES-128-GCM)	2 bytes
Epoch numbers	Bob tracks internally	3 (first app data epoch)	8 bytes
Packet number ranges	Bob tracks per direction	write: 0..15, read: 0..3	48 bytes
Local CIDs (8)	Bob issued to Alice	0x7f8a3c1b9e2d4f06a5c8, ...	162 bytes
Remote CIDs (1)	Alice issued to Bob	0x2c5f8b1a7d3e9046c8a2	32 bytes
Alice's address	From the network path	141.217.168.127:50000	7 bytes
QUIC Version	Negotiated	0x00000001 (v1)	4 bytes

What Charlie does with it:

Charlie receives 445 bytes via TCP/Redis/HTTP.

Step 1: Import the raw secret bytes as NSS key objects

```
write_secret = hkdf::import_key(0xa11af9f05531f856...)
read_secret  = hkdf::import_key(0x9e40646ce79a7f9d...)
```

Step 2: Re-derive AEAD and HP keys (IDENTICAL to Bob's)

```
write_aead_key = HKDF-Expand(write_secret, "quic key", 16)
                = 0xfd8c7da9de1b2da4d2ef9fd5188922d0 ← same as Bob!
write_iv       = HKDF-Expand(write_secret, "quic iv", 12)
                = 0x02f6180e4f4aa456d7e8a389         ← same as Bob!
write_hp       = HKDF-Expand(write_secret, "quic hp", 16)
                = 0xb7f6f021453e43b0b2c25989d0a31207 ← same as Bob!
```

(Same for read direction)

Step 3: Set up connection state

```
conn.state = Confirmed (skip handshake)
conn.local_cids = [8 CIDs from Bob]
conn.remote_cids = [Alice's CID]
conn.path = Alice's IP:port ↔ Charlie's IP:port
```

Charlie is now a PERFECT CLONE of Bob's connection.

Charlie never talked to Alice. Charlie never did Diffie-Hellman. Charlie never saw the certificate. But Charlie has byte-identical encryption keys. Alice cannot tell the difference when Charlie responds to her PATH_CHALLENGE.

5. Who Knows What? (Complete Picture)

Secret / Value	Alice	Bob	Charlie	Eve
Alice's X25519 private key	YES	No	No	No
Bob's X25519 private key	No	YES	No	No
Both X25519 public keys	Yes	Yes	No	Yes (on wire)
ECDHE shared_secret	YES	YES	No	No
master_secret	YES	YES	No	No
client_app_secret (32 bytes)	YES	YES	YES (from Bob)	No
server_app_secret (32 bytes)	YES	YES	YES (from Bob)	No
AEAD keys / IV / HP keys	YES	YES	YES (re-derived)	No
Can decrypt Alice's packets?	YES	YES	YES	No
Can encrypt responses to Alice?	N/A	YES	YES	No
Connection IDs	YES	YES	YES (from Bob)	Yes (on wire)
Handshake transcript	YES	YES	No	Partial

Charlie knows LESS than Bob: he doesn't have the handshake transcript, the DH shared secret, or the master secret. He only has the final application secrets. But that's enough -- the application secrets are all you need to encrypt and decrypt app data.

Eve knows NOTHING useful. She can see public keys and CIDs on the wire, but cannot compute any secrets. Even if she captures the 445-byte transfer between Bob and Charlie (e.g., on the LAN), she gets the secrets. That's why the transfer channel **MUST** be encrypted in production.

6. The Complete Recipe (One Page Summary)

RECIPE: How to Build OUIC Application Secrets

INGREDIENTS:

```
From Alice: private_key_a (32 bytes, random, SECRET)
From Bob:   private_key_b (32 bytes, random, SECRET)
From RFC:   QUIC v1 salt, TLS labels (public constants)
```

STEP 1 ■ Key Exchange (Diffie-Hellman)

```
public_a = X25519(private_a, basepoint)    [Alice computes, sends to Bob]
public_b = X25519(private_b, basepoint)    [Bob computes, sends to Alice]
shared   = X25519(private_a, public_b)      [Both compute independently]
         = X25519(private_b, public_a)      [Same result!]
```

STEP 2 ■ Handshake Secret

```
early_secret      = HKDF-Extract(salt=0, IKM=zeros)
derived_1         = HKDF-Expand(early_secret, "derived")
handshake_secret  = HKDF-Extract(salt=derived_1, IKM=shared)
```

STEP 3 ■ Master Secret

```
derived_2      = HKDF-Expand(handshake_secret, "derived")
master_secret  = HKDF-Extract(salt=derived_2, IKM=zeros)
```

STEP 4 ■ Application Secrets (what we export!)

```
transcript      = SHA-256(all handshake messages)
client_secret   = HKDF-Expand(master, "c ap traffic", transcript)
server_secret   = HKDF-Expand(master, "s ap traffic", transcript)
```

STEP 5 ■ Actual Keys (derived on both Bob AND Charlie)

```

aad_key = HKDF-Expand(secret, "quic key", 16)    [same input = same key]
iv       = HKDF-Expand(secret, "quic iv", 12)     [same input = same iv]
hp_key   = HKDF-Expand(secret, "quic hp", 16)     [same input = same hp]

```

```
EXPORT: Bob sends client_secret + server_secret (64 bytes) to Charlie
```

IMPORT: Charlie runs Step 5 and gets IDENTICAL keys

Total exported: 64 bytes of secrets + 381 bytes of metadata = 445 bytes
Total security: Entire connection. Cannot be forged. Cannot be reversed

The 32-byte application secret is the distilled essence of: Alice's randomness + Bob's randomness + their Diffie-Hellman exchange + the hash of every handshake message. All compressed into 32 bytes by HKDF. Charlie only needs these 32 bytes (per direction) to become a perfect substitute for Bob.